

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S2
DATE PERFORMED:	July 29, 2026	DATE SUBMITTED:	August 5, 2026
NAME:	Johrel Louie T. Silao	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (std::stack).

```

1  #include <iostream>
2  #include <stack>
3
4  int main () {
5
6      // Testing ground of Stack
7      std::stack<int> stack1;
8      std::stack<int> stack2;
9      std::cout<<"=====Testing the Stack STL===== " << std::endl;
10
11     // isEmpty
12     std::cout<<"Is the stack empty? " << stack1.empty() << std::endl;
13
14     // Push
15     std::cout << "=====Push Stack===== " << std::endl;
16     stack1.push(10);
17     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
18     stack1.push(9);
19     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
20     stack1.push(8);
21     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
22     stack1.push(7);
23     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
24     stack1.push(6);
25     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
26     stack1.push(5);
27     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
28     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
29     std::cout << std::endl;
30
31     // Pop
32     stack1.pop();
33     std::cout << "=====Pop Stack===== " << std::endl;
34     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
35     std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
36     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
37
38     // Eemplace
39     std::cout << "=====Emplace Stack===== " << std::endl;
40     stack1.emplace(1); // Replaces the top value of the stack
41     std::cout << "The top of the stack is: " << stack1.top() << std::endl;
42     std::cout << "The size of the stack is: " << stack1.size() << std::endl;
43
44     // Swap
45     stack2.push(4);
46     stack2.push(3);
47     stack2.push(2);
48     stack1.swap(stack2); // Swaps the contents of stack1 and stack2
49
50     //Swapped contents of stack 1 and stack 2
51     std::cout << "=====Swap Stack===== " << std::endl;
52     std::cout << "The top of the stack 1 is: " << stack1.top() << std::endl;
53     std::cout << "The top of the stack 2 is: " << stack2.top() << std::endl;
54
55     std::cout << "The size of the stack 1 is: " << stack1.size() << std::endl;
56     std::cout << "The size of the stack 2 is: " << stack2.size() << std::endl;
57     return 0;
58 }
```

Figure 1: Code Snippet of ILO A Implementing a stack using the C++ STL (std::stack).

Analysis of Figure 1:

- This code snippet shows the basic implementation of the template library 'std::stack' where it tests multiple member functions. It pushes the elements 10 - 5 for stack1, which proves the Last-In, First-Out (LIFO) behavior by displaying the top element after each implementation and checks the size of the stack 'size()', as well as the emptiness of the stack 'empty()', to access the top element 'top()', replaces the value of the stack 'emplace()', and to swap all the contents and size between both stack objects 'stack.swap()'.

```

=====Testing the Stack STL=====
Is the stack empty? 1
=====Push Stack=====
The top of the stack is: 10
The top of the stack is: 9
The top of the stack is: 8
The top of the stack is: 7
The top of the stack is: 6
The top of the stack is: 5
The size of the stack is: 6

=====Pop Stack=====
The top of the stack is: 6
Is the stack empty? 0
The size of the stack is: 5
=====Emplace Stack=====
The top of the stack is: 1
The size of the stack is: 6
=====Swap Stack=====
The top of the stack 1 is: 2
The top of the stack 2 is: 1
The size of the stack 1 is: 3
The size of the stack 2 is: 6

```

Figure 2: Output Snippet of ILO A Implementing a stack using the C++ STL (**std::stack**).

Analysis of Figure 2:

- Based on the output of figure 2, it shows the application of the C++ STL 'std::stack' abiding to the Last-In, First-Out (LIFO) behavior. At first, the stack is empty, after pushing six values onto the stack, it has a total size of 6 with the element of '5' as the top element. Following the popping operation, that removes the element '5', which leaves the element 6 as the top element, then the emplace function, replaces directly the value on top to increase the size back to 6, while the swap function swaps the contents and sizes between two stack objects.

ILO B.1: Implement a stack using a Static Array.

```

1  #include <iostream>
2
3      // Global Declarations
4  #define maxCap 10
5      int choice;
6      int stackArr [maxCap];
7      int top = -1, newData;
8
9      // Prototype Functions
10     void push();
11     void pop();
12     void Top();
13     bool isEmpty();
14     bool isFull();
15     void displayStack();
16
17 int main() {
18
19     // Main Driver
20     while (true) {
21         std::cout << "=====Stack Operations:===== " << std::endl;
22         std::cout << "1. Push Stack" << std::endl;
23         std::cout << "2. Pop Stack" << std::endl;
24         std::cout << "3. Top Stack" << std::endl;
25         std::cout << "4. isEmpty Stack" << std::endl;
26         std::cout << "5. isFull Stack" << std::endl;
27         std::cout << "6. Display the stack" << std::endl;
28         std::cout << "7. Exit the program" << std::endl;
29         std::cout << "===== " << std::endl;
30         std::cin >> choice;
31
32         if (choice == 7) {
33             std::cout << "Exiting the Program!" << std::endl;
34             break;
35         }
36
37         switch (choice) {
38             case 1: push();
39                 break;
40             case 2: pop();
41                 break;
42             case 3: Top();
43                 break;
44             case 4: std::cout << "Is the stack empty? " << isEmpty() << std::endl;
45                 break;
46             case 5: std::cout << "Is the stack full? " << isFull() << std::endl;
47                 break;
48             case 6: displayStack();
49                 break;
50             default: std::cout << "Invalid Choice" << std::endl;
51         }
52     }
53
54     return 0;
55 }

```

Figure 3: Code Snippet of ILO B.1 Implementing a stack using a Static Array.

Analysis of Figure 3:

- In this figure, it sets up the foundation of ILO B.1 for implementing a stack using a Static Array. It first defines a maximum stack capacity of 10 'maxCap', an array that stores the values 'stackArr', tracks the stack pointer 'top = -1', and declares functions for the stack operations. The main function uses a menu interface for the user with a 'while' loop, paired with a 'switch' case statement.

```

57 // Function Definition
58
59 bool isEmpty() {
60     // Error checking
61     if (top == -1) return true;
62     return false;
63 }
64
65 bool isFull() {
66     if (top == maxCap - 1) return true;
67     return false;
68 }
69
70 void push() {
71     // Error checking
72     if (isFull()) {
73         std::cout << "Stack Overflow" << std::endl;
74         return;
75     }
76     // Pushing to the stack
77     std::cout << "Enter a new value: " << std::endl;
78     std::cin >> newData;
79     stackArr[++top] = newData;
80 }
81
82 void pop() {
83     // Error checking
84     if (isEmpty()) {
85         std::cout << "Stack Underflow" << std::endl;
86         return;
87     }
88
89     // Displays the "popped" value of the stack
90     std::cout << "Value Popped: " << stackArr[top] << std::endl;
91
92     // Decrement the top value from the stack
93     top--;
94 }
95
96 void Top () {
97     // Error Catching
98     if (isEmpty()) {
99         std::cout << "The stack is empty!" << std::endl;
100         return;
101     }
102
103     // Checks the top value:
104     std::cout << "Top Element: " << stackArr[top] << std::endl;
105 }

```

Figure 4: Code Snippet of ILO B.1 Implementing a stack using a Static Array

Analysis of Figure 4:

- In this figure, it implements the operations for the stack operations, including the error checking, pushing the stack, popping the stack, and displaying the top element. The functions 'isEmpty()' and 'isFull()' checks where the top element is equal to -1, which will return a boolean output to prevent an execution error. The 'push()' checks if the stack is overflowing before inserting a new element, while 'pop()' checks the stack underflow before displaying the top element and "removing" the top element. Additionally, the 'top()' function checks the current top element and provides the stack that is not empty.

```

106
107 //Displays the elements of the stack
108 void displayStack() {
109     if (isEmpty()) {
110         std::cout << "The stack is empty!" << std::endl;
111     }
112     for (int i = top; i >= 0; i--) {
113         std::cout << stackArr[i] << ",";
114     }
115     std::cout << std::endl;
116 }

```

Figure 5: Code Snippet of ILO B.1 Implementing a stack using a Static Array.

Analysis of Figure 5:

- This figure defines the function 'displayStack()' where it prints all the stored elements. It first notifies the user that there are no elements available to display using the 'isEmpty()' function. If there are elements available in the array, it prints each element separated by a comma. Thus completes this section of the activity.

[illegible]

Push Stack

```

=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
6
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
7
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
8
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
9
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Enter a new value:
10
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====
1
Stack Overflow
=====Stack Operations:=====
1. Push Stack
2. Pop Stack
3. Top Stack
4. isEmpty Stack
5. isFull Stack
6. Display the stack
6. Exit the program
=====

```

It prints the execution of the program, using the option 1 "Push Stack," triggers a prompt asking the user to enter a new value, after each input, it will be placed onto the top of the static array stack. In this series of insertion of values, it demonstrates the LIFO data structure before it reaches the max capacity of the array. When the user attempts to insert another value in the stack, the program executes the 'isFull()' function and prints the "Stack Overflow."

Pop Stack	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 6. Exit the program ===== 2 Value Popped: 10 </pre>	It shows the execution of the stack's removal when option 2 "Pop Stack" was chosen from the menu interface. Option 2, executes the 'pop()' function, it checks the stack underflow before displaying the top element and "removing" the top element which correctly displays the "Value Popped: 10."
Top Stack	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 6. Exit the program ===== 3 Top Element: 9 </pre>	It executes the option 3 using the 'top()' function, where it checks and displays the current top element of the stack. Which correctly displays the "Top Element: 9."
isEmpty Stack	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 6. Exit the program ===== 4 Is the stack empty? 0 </pre>	It executes the option 4 using the 'isEmpty()' function, where it checks if the stack pointer top is equal to -1. The function will return false, because the stack contains the elements 1 - 9.
isFull Stack	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 6. Exit the program ===== 5 Is the stack full? 0 </pre>	This executes the option 5 using the 'isFull()' function, where it checks whether the stack is full or not.

Display the Stack	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 7. Exit the program ===== 6 9,8,7,6,5,4,3,2,1, </pre>	It executes the option 6 using the 'displayStack()' function, where it prints all the stored elements.
Exiting of Program	<pre> =====Stack Operations:===== 1. Push Stack 2. Pop Stack 3. Top Stack 4. isEmpty Stack 5. isFull Stack 6. Display the stack 7. Exit the program ===== 7 Exiting the Program! PS C:\Users\TIPQC> </pre>	It executes the option 7, triggering the last switch case statement in the program, forcing the user to exit the program, leaving a message "Exiting the Program!"

Table 1: Output of ILO B.1 Implementing a stack using a Static Array.

```

1  #include <iostream>
2  #include "stackList.h"
3
4  int main() {
5
6      std::cout << "---- PUSHING 'C' 'P' 'E' ONTO THE STACK ----\n";
7
8      // Pushing
9      push('C');
10     push('P');
11     push('E');
12
13     // Display the current top
14
15     Top<char>; // Top should be 'E'
16     displayALL<char>();
17     std::cout << "\n ---- POPPIING ALL ELEMENT ----\n";
18
19     while(!isEmpty<char>()) {
20         std::cout<<"Popped: "<< pop<char>() <<std::endl;
21     }
22
23     std::cout << "\n ---- TESTING THE UNDERFLOW ---- \n";
24     pop<char>();
25
26     return 0;
27 }

```

Figure 6: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 6:

- In this figure, it contains the main function that shows the application of a linked list header. The program pushes three (3) characters 'C', 'P', 'E' on the stack before displaying the top element and prints all the stored nodes. To be followed by the 'while (!isEmpty<char>())' loop to continuously pop and print the elements until the stack is empty. Then it will test if the stack is empty which will verify that the stack underflow mechanism works.

```

1  #ifndef STACKLIST_H
2  #define STACKLIST_H
3
4  #include <iostream>
5
6  template <typename T>
7  class Node {
8      public:
9          T data; // VALUE WE NEED TO STORE
10         Node* next; // POINTER TO THE NEXT NODE
11     };
12
13     // Global pointer for the top of the stack
14     template <typename T>
15     Node<T>* head = nullptr;
16
17     // ----- Operations for the stack -----//
18
19     // Push
20     template <typename T>
21     void push(T newData) {
22         // Create a new node and assign the newData
23         Node<T>* newNode = new Node<T>();
24         newNode->data = newData;
25
26         // New node points to current top, then becomes the new top
27         newNode->next = head<T>;
28         head<T> = newNode;
29     }

```

Figure 7: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 7:

- In this figure, it shows the definitions and initial implementations of the linked list stack structure. The 'Node<T>' class is defined with a data type of T and a nullptr to reference the node in the list. The global pointer will be initialized to nullptr because that will serve as a top pointer for the linked stack. Finally, the function 'push(T newData)' function will allocate a new node on the heap, then it will assign its data and points its next pointer to the current head and then it will reassign the head to point to the new node.

```

31 // Pop
32
33 template <typename T>
34 T pop() {
35     // Check if the stack is empty
36     if (head<T> == nullptr) {
37         std::cout << "Stack Underflow" << std::endl;
38         return T();
39     }
40
41     // Save the current top node
42     Node<T>* temp = head<T>;
43     // Store its data
44     T tempVal = temp->data;
45
46     // Move the top pointer to the next node
47     head<T> = head<T>->next;
48
49     // Free the removed node
50     delete temp;
51
52     // Return the removed value
53     return tempVal;
54 }
55
56 // isEmpty
57 template <typename T>
58 bool isEmpty() {
59     return head<T> == nullptr;
60 }
61
62 // Top
63 template <typename T>
64 void Top() {
65     // Error Catching:
66     if (head<T> == nullptr) {
67         std::cout << "Stack is empty." << std::endl;
68         return;
69     }
70     // Display the top element
71     std::cout << "Top element: " << head<T>->data << std::endl;
72 }

```

Figure 8: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 8:

- This figure will implement the stack, including the popping function, checking of an empty list, and displaying the top element. The function 'pop()' will protect against the underflow, returning T() if true, or else, it will save the top node's data, it will now proceed to head to head->next, frees the unlinked node with delete, and returns the stored value.

```

74 // Display All
75 template <typename T>
76 void displayALL() {
77     // Error Catching:
78     if (head<T> == nullptr) {
79         std::cout << "Stack Underflow\n" << std::endl;
80         return;
81     }
82
83     // Traverse the stack and display all elements
84     Node<T>* temp = head<T>;
85     std::cout << "Stack elements: \n";
86     while (temp != nullptr) {
87         std::cout << temp->data << " \n";
88         temp = temp->next;
89     }
90     std::cout << std::endl;
91 }
92
93 #endif

```

Figure 9: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 9:

- In this figure, it defines the 'displayAll()' function that will traverse and display all the elements in the linked stack. It will first perform an error checking if 'head<T> == nullptr', which will then print the message "Stack Underflow" and return if there are no existing nodes.

```

--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---
Stack elements:
E
P
C

--- POPPIING ALL ELEMENT ---
Popped: E
Popped: P
Popped: C

--- TESTING THE UNDERFLOW ---
Stack Underflow: Cannot pop from an empty stack.

```

Figure 10: Output Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 10:

- In this figure, it shows the output of the code structure of ILO B.2, after pushing the characters 'C', 'P', and 'E', it will call the 'displayAll()' function and prints the elements in a vertically reversed order, that abides the LIFO behavior.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

- a. Stack using Arrays
- b. Stack using Linked Lists
- c. (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {
 - a) If the character read is not a symbol to be balanced, ignore it.
 - b) If the character is an opening symbol, push it onto the stack.
 - c) If it is a closing symbol:
 - i) Report an error if the stack is empty.
 - ii) Otherwise, pop the stack.
 - d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Self-Checking:

For the following cases, complete the table using the code you created.

```

1  #ifndef STACKLIST_H
2  #define STACKLIST_H
3
4  #include <iostream>
5
6  template <typename T>
7  class Node {
8  public:
9      T data; // VALUE WE NEED TO STORE
10     Node* next; // POINTER TO THE NEXT NODE
11 };
12
13 // Global pointer for the top of the stack
14 template <typename T>
15 Node<T>* head = nullptr;
16
17 // ----- Operations for the stack -----//
18
19 // Push
20 template <typename T>
21 void push(T newData) {
22     // Create a new node and assign the newData
23     Node<T>* newNode = new Node<T>();
24     newNode->data = newData;
25
26     // New node points to current top, then becomes the new top
27     newNode->next = head<T>;
28     head<T> = newNode;
29 }

```

Figure 11: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 11:

- In this figure, it shows the definitions and initial implementations of the linked list stack structure. The 'Node<T>' class is defined with a data type of T and a nullptr to reference the node in the list. The global pointer will be initialized to nullptr because that will serve as a top pointer for the linked stack. Finally, the function 'push(T newData)' function will allocate a new node on the heap, then it will assign its data and points its next pointer to the current head and then it will reassign the head to point to the new node.

```

31 // Pop
32
33 template <typename T>
34 T pop() {
35     // Check if the stack is empty
36     if (head<T> == nullptr) {
37         std::cout << "Stack Underflow" << std::endl;
38         return T();
39     }
40
41     // Save the current top node
42     Node<T>* temp = head<T>;
43     // Store its data
44     T tempVal = temp->data;
45
46     // Move the top pointer to the next node
47     head<T> = head<T>->next;
48
49     // Free the removed node
50     delete temp;
51
52     // Return the removed value
53     return tempVal;
54 }
55
56 // isEmpty
57 template <typename T>
58 bool isEmpty() {
59     return head<T> == nullptr;
60 }
61
62 // Top
63 template <typename T>
64 void Top() {
65     // Error Catching:
66     if (head<T> == nullptr) {
67         std::cout << "Stack is empty." << std::endl;
68         return;
69     }
70     // Display the top element
71     std::cout << "Top element: " << head<T>->data << std::endl;
72 }

```

Figure 12: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 12:

- This figure will implement the stack, including the popping function, checking of an empty list, and displaying the top element. The function 'pop()' will protect against the underflow, returning T() if true, or else, it will save the top node's data, it will now proceed to head to head->next, frees the unlinked node with delete, and returns the stored value.

```
74 // Display All
75 template <typename T>
76 void displayALL() {
77     // Error Catching:
78     if (head<T> == nullptr) {
79         std::cout << "Stack Underflow\n" << std::endl;
80         return;
81     }
82
83     // Traverse the stack and display all elements
84     Node<T>* temp = head<T>;
85     std::cout << "Stack elements: \n";
86     while (temp != nullptr) {
87         std::cout << temp->data << " \n";
88         temp = temp->next;
89     }
90     std::cout << std::endl;
91 }
92
```

Figure 13: Code Snippet of ILO B.2 Stacks Using Linked Lists.

Analysis of Figure 13:

- In this figure, it defines the 'displayAll()' function that will traverse and display all the elements in the linked stack. It will first perform an error checking if 'head<T> == nullptr', which will then print the message "Stack Underflow" and return if there are no existing nodes.

```

94 | | | // ----- Implementation of Functions in Supplementary Activity ----- //
95 |
96 | // Opening Symbol
97 | inline bool isOpen(char c) {
98 |     return (c == '(' || c == '[' || c == '{');
99 | }
100 |
101 | // Closing Symbol
102 | inline bool isClose(char c) {
103 |     return (c == ')' || c == ']' || c == '}');
104 | }
105 |
106 | // Symbols Matched
107 | inline bool isMatched(char openChar, char closeChar) {
108 |     switch (openChar) {
109 |         case '(': return closeChar == ')';
110 |         case '[': return closeChar == ']';
111 |         case '{': return closeChar == '}';
112 |         default: return false;
113 |     }
114 | }
115 |
116 | // Symbols Balanced
117 | inline bool isBalanced(const std::string& expression) {
118 |     // Reset global stack state
119 |     while (!isEmpty<char>()) {
120 |         pop<char>();
121 |     }
122 |
123 |     for (size_t i = 0; i < expression.length(); ++i) {
124 |         char currentChar = expression[i];
125 |
126 |         if (isOpen(currentChar)) {
127 |             push<char>(currentChar);
128 |         }
129 |         else if (isClose(currentChar)) {
130 |             // Unmatched closing symbol found when stack is empty
131 |             if (isEmpty<char>()) {
132 |                 return false;
133 |             }
134 |
135 |             char lastOpened = pop<char>();
136 |             if (!isMatched(lastOpened, currentChar)) {
137 |                 return false;
138 |             }
139 |         }
140 |     }
141 |
142 |     return isEmpty<char>();
143 | }
144 |
145 | #endif

```

Figure 14: Code Snippet of ILO C

Analysis of Figure 14:

- In this figure, it shows the implementation in the supplementary activity for evaluating expressions that contains parentheses, brackets and curly brackets. The functions; isOpen(), isClose(), and isMatched() determines what type of bracket/s is involved and it will check whether the given problem matches correctly with a closing character. The function 'isBalanced()' will first reset the global stack, then it will iterate character-by-character through the inputted string. After analyzing the entire string, the function will now return to the 'isEmpty<char>()' to make sure that all opened symbols were closed and matched.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	<pre>===== BALANCING OF SYMBOL ===== Enter an expression: (A+B)+(C-D) Result: Expression is Balanced. =====</pre>	This expression is a valid Y because every opening parentheses has a closing parentheses in order.
((A+B)+(C-D)	N	<pre>Would you like to continue? (Y/N): Y Enter an expression: ((A+B)+(C-D) Result: Expression is Unbalanced. =====</pre>	This expression is a valid N because it has an unmatched opening parentheses.
((A+B)+[C-D])	Y	<pre>Would you like to continue? (Y/N): Y Enter an expression: ((A+B)+[C-D]) Result: Expression is Balanced. =====</pre>	This expression is a valid Y because both parentheses and square brackets are matched and balanced in an enclosed parentheses.
((A+B)+[C-D])}	N	<pre>Would you like to continue? (Y/N): Y Enter an expression: ((A+B)+[C-D])} Result: Expression is Unbalanced. ===== Would you like to continue? (Y/N): N Exiting the Program!</pre>	This expression is a valid N because it doesn't match the parentheses and square brackets.

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?
 - Thinking about what to use in internal representation directly will change how a stack will operate, its performance characteristics, and how safely it can be implemented and used. For example, a static array-based stack, the elements are stored in a fixed storage of memory that is managed by an integer tracking the top index. With this, the C++ Standard Template Library works as a high-level container adapter that combines the dynamical flexibility and performance optimizations.

Conclusion:

In this activity, I learned how the “Stack” implementation application works by Last-In, First-Out (LIFO) data structures where elements are added and removed strictly from the top. To prevent errors, I used `isEmpty()` to return true when the top index is -1, which shows that there are no elements left to access or remove, while `isFull()` returns true when the top index reaches the max cap size, to signal that there are no more elements to store. The `push()` stores a new value, while the `pop()` decrements or removes the value. To check the recent value that is added, the `top()` is used. Finally the `displayStack()` checks if the stack contains elements. For the Linked List-Based Stack Implementation, it works by a dynamic heap allocation where each element is represented by a node that contains data and a pointer to the next dynamic node. As for the supplementary activity of Symbol Balancing, the logic was evaluated through an expression symbol-balancing that will process string characters, opening, closing and matching the symbols. Based on the experiment, balanced symbols will only appear correctly if the symbols meet the condition of properly matched while the unbalanced symbols will only appear incorrectly if the symbols meet the condition incorrectly due to unmatched symbols or incorrectly placed symbols. Doing this hands-on activity is really lengthy and very tiring. It took me a long time to finish this activity, I encountered a lot of errors because of incorrect variable names and incorrect placement of pointers or the asterisk. In the end, I was able to finish the activity by collaborating with other groups or people around me, asking for guidance and knowledge.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.